Claude Solution Architect Foundation — Master Terminology & Concepts Reference

This document is a practical learning reference for Claude/LLM systems engineering concepts commonly appearing in:

- Claude ecosystems
- MCP systems
- AI agents
- RAG pipelines
- Prompt engineering
- Production AI systems
- Claude Solution Architect Foundation preparation

---

1. Prompt Engineering Concepts

Term	Description	Purpose	Usage	Example	Important Learning Insight
Zero-shot prompting	Asking the model without examples	Fast/simple prompting	General chat, simple tasks	“Explain GraphQL.”	Lowest reliability for structured tasks
One-shot prompting	Provide one example before task	Teach simple pattern	Formatting tasks	One JSON example before extraction	Often enough for simple workflows
Few-shot prompting	Multiple examples inside prompt	Improve consistency and structure	Extraction, coding, classification	Example input/ output pairs	Good examples matter more than long prompts
Chain-of-thought	Encourage step-by-step reasoning	Improve reasoning accuracy	Math, planning, logic	“Think step by step”	Can increase latency and token cost
Structured prompting	Highly organized prompt format	Reduce ambiguity	Enterprise AI systems	Sections/ rules/ templates	Structure improves first-pass success

Term	Description	Purpose	Usage	Example	Important Learning Insight
Role prompting	Assign model a role/persona	Control tone/ behavior	Support bots, experts	“You are a security architect”	Weak roles rarely help without constraints
Prompt templates	Reusable prompt patterns	Standardization	Production systems	Jinja-style templates	Critical for scalability
Delimiters	Explicit content boundaries	Parsing reliability	Tool calls, JSON	`<DATA>`	Weak delimiters create parsing bugs
Stop sequences	Tokens/strings stopping generation	Prevent unwanted continuation	APIs, structured outputs	`<END>`	Very important for reliability
Recursive summarization	Multi-layer summarization strategy	Compress huge context	Long docs, memory systems	Chunk → summarize → summarize summaries	Information degrades each layer
Context injection	Insert external knowledge/ context	Ground responses	RAG systems	Inject retrieved docs	Poor retrieval causes hallucinations
Prompt chaining	Multi-step prompts connected together	Complex workflows	Agents/ pipelines	Extract → validate → summarize	Chains increase orchestration complexity

2. Inference & Sampling Parameters

Term	Description	Purpose	Usage	Example	Important Learning Insight
Temperature	Controls randomness intensity	Creativity vs determinism	Coding, writing	`temperature: 0.2`	High temperature increases hallucinations

Term	Description	Purpose	Usage	Example	Important Learning Insight
top_p	Nucleus sampling threshold	Restrict candidate token pool	Structured outputs	`top_p: 0.3`	Lower values improve consistency
max_tokens	Max response length	Cost/control	APIs	`max_tokens: 4000`	Too low truncates reasoning
Streaming	Incremental output delivery	Real-time UX	Chat apps	`stream: true`	Harder to orchestrate tools
Sampling	Token selection strategy	Control generation behavior	All inference	temp + top_p	Reliability depends heavily on sampling
Deterministic generation	Minimized randomness	Repeatable outputs	Extraction/ testing	Low temp + low top_p	Essential for enterprise reliability
Token	Small text unit processed by model	Fundamental billing/ context unit	All LLMs	“hello” → several tokens	Costs scale with tokens
Context window	Maximum processable tokens	Long memory	Large chats/docs	200k+ token contexts	Bigger context ≠ perfect recall

3. Tool Use & MCP Concepts

Term	Description	Purpose	Usage	Example	Important Learning Insight
Tool use	LLM invoking external functions	Extend capability	Agents/APIs	Weather lookup	Tools matter more than model sometimes

Term	Description	Purpose	Usage	Example	Important Learning Insight
tool_choice	Controls tool usage behavior	Reliability/ control	Claude API	`{"type":"auto"}`	`any` forces tool usage
Tool annotations	Structured tool descriptions	Improve tool selection	Agents	JSON schemas	Weak annotations break systems
Function calling	Structured external invocation	Safe integrations	APIs	Calculator tool	Hallucinated arguments are common
MCP	Model Context Protocol	Standardized AI-tool integration	Claude ecosystems	MCP servers	Mostly orchestration infrastructure
MCP server	External provider of tools/ resources	Extend context/ tooling	Enterprise integrations	GitHub MCP	Security matters heavily
Resources	External contextual assets	Dynamic grounding	MCP	File/document resources	Poor resource management bloats context
Tool schema	JSON structure for tools	Reliable parsing	APIs	input_schema	Critical for first-pass success
Structured outputs	Constrained response formats	Parsing reliability	JSON APIs	Strict JSON output	Models still occasionally break schema

4. Agent Architecture Concepts

Term	Description	Purpose	Usage	Example	Important Learning Insight
Agent	LLM system executing tasks/tools	Workflow automation	Coding/ research	AI coding agent	Most agents are orchestration systems

Term	Description	Purpose	Usage	Example	Important Learning Insight
Orchestration	Coordinating AI workflows	Multi-step execution	Enterprise AI	Planner + executor	Harder than prompting
Planner agent	Decides workflow strategy	Task decomposition	Multi-agent systems	Create execution plan	Planning quality affects reliability
Executor agent	Performs concrete actions	Task execution	Automation systems	Tool calls	Executors require strong validation
Synthesis subagent	Combines outputs from multiple agents	Final coherence	Multi-agent systems	Merge research outputs	Often hardest part of architecture
Reflection	Self-review/ reasoning loop	Improve outputs	Advanced agents	Critique own answer	Can cause loops/token explosion
Hooks	Trigger points for logic	Extend workflows	Tool events	pre_response_hook	Too many hooks create chaos
Gates	Validation checkpoints	Safety/ reliability	Production systems	Confidence threshold	Weak gates destroy trust
Retries	Repeat failed operations	Resilience	APIs/ agents	Retry tool failure	Blind retries create loops
Workflow	Structured execution pipeline	Automation	Enterprise systems	Extract → validate → save	Complexity scales rapidly
Multi-agent systems	Multiple specialized agents	Parallel specialization	Research/ coding	Planner + workers	Often overhyped marketing
context:fork	Duplicate context branch	Parallel exploration	Agents	Try multiple strategies	Merge complexity becomes difficult
fork_session	Branch existing session	Alternate workflows	Claude workflows	Session branching	Shared-state bugs common

Term	Description	Purpose	Usage	Example	Important Learning Insight
Task decomposition	Splitting large problems	Scalability	Complex workflows	Break coding tasks apart	Poor decomposition ruins results

5. Retrieval & Knowledge Systems

Term	Description	Purpose	Usage	Example	Important Learning Insight
RAG	Retrieval-Augmented Generation	Inject external knowledge	Enterprise AI	Retrieve docs before answer	Retrieval quality determines accuracy
Embeddings	Numeric semantic representations	Similarity search	Vector DBs	Sentence embeddings	Embeddings are not reasoning
Vector database	Stores embeddings	Semantic retrieval	RAG	Pinecone/ Weaviate	Garbage embeddings → garbage retrieval
Semantic search	Meaning-based retrieval	Better relevance	Search systems	Similarity search	Retrieval tuning matters heavily
Chunking	Splitting documents into pieces	Retrieval optimization	RAG	500-token chunks	Bad chunking hurts retrieval badly
Hybrid retrieval	Multiple retrieval strategies combined	Better accuracy	Enterprise search	BM25 + embeddings	More complex but stronger
Reranking	Reordering retrieved results	Improve relevance	RAG pipelines	Cross-encoder reranking	Often dramatically improves results
Citation	Source reference	Traceability	AI answers	Source links	Citation ≠ correctness

Term	Description	Purpose	Usage	Example	Important Learning Insight
citation_id	Internal source identifier	Evidence tracking	RAG systems	`doc_22`	Mostly machine-readable

6. Context & Memory Management

Term	Description	Purpose	Usage	Example	Important Learning Insight
Context pruning	Removing low-value history	Token optimization	Long chats	Drop stale messages	Poor pruning loses critical info
Memory compression	Summarizing history	Long-running agents	Persistent systems	Recursive summaries	Compression loses detail
Persistent memory	Long-term stored state	Personalization	Assistants	Stored preferences	Requires privacy controls
Session state	Runtime conversational state	Continuity	Chat systems	Current task state	Corruption causes weird behavior
Token budgeting	Managing token usage	Cost/ performance	Production systems	Context allocation	Often overlooked by beginners
Memory files	Stored AI instructions/ context	Claude Code	CLAUDE.md	Project memory	Important for coding workflows

7. Evaluation & Reliability

Term	Description	Purpose	Usage	Example	Important Learning Insight
First-pass success	Correct result on first attempt	Reliability	Enterprise AI	No retries needed	Good prompts/ tooling critical
Evals	Systematic AI testing	Measure quality	Production AI	Benchmark suites	Most teams lack proper evals
Hallucination	Confidently incorrect generation	Major AI risk	All LLM systems	Fake citations	Cannot be fully eliminated
Confidence scoring	Estimated correctness probability	Automation decisions	Extraction systems	95% confidence	Confidence is often poorly calibrated
Calibration	Alignment between confidence and reality	Reliability	Production AI	90% confidence ≈ 90% accuracy	Critical for automation
Benchmark contamination	Model exposed to eval data	Invalid testing	AI research	Training leakage	Huge hidden problem in AI evals
strict_novelty	Enforce highly distinct outputs	Prevent duplication	Test generation	Novel exam questions	Too much novelty hurts realism
coverage-report	Test coverage metrics and execution visibility	Measure tested code paths	CI/CD, QA pipelines, automated testing	HTML or CLI coverage reports	High coverage does NOT guarantee good tests
coverage-report	Test coverage metrics	QA visibility	CI/CD	Coverage percentages	High coverage ≠ good tests
Regression testing	Verify old behavior still works	Stability	Deployments	Re-run eval suites	Essential for production systems

Term	Description	Purpose	Usage	Example	Important Learning Insight
Reliability metrics	System quality indicators	Monitoring	Enterprise AI	Accuracy/ latency/ failure rate	Metrics without context mislead

8. Extraction & Structured Data Systems

Term	Description	Purpose	Usage	Example	Important Learning Insight
Extraction system	Structured data extraction pipeline	Automation	Invoices/ resumes	OCR → JSON	Production extraction is difficult
OCR	Optical Character Recognition	Read scanned text	PDFs/ images	Invoice scanning	OCR errors cascade downstream
Schema validation	Verify output structure	Reliability	JSON systems	Validate required fields	Essential for enterprise AI
Entity extraction	Identify specific data entities	Structured parsing	NLP systems	Names/ dates/ emails	Ambiguity causes failures
Classification	Categorize inputs	Routing/ automation	Support systems	Spam detection	Poor labels reduce reliability
Confidence thresholds	Automation gating limits	Human review routing	Extraction	Auto-approve >95%	Threshold tuning is critical

9. Claude Code & Workflow Commands

Term	Description	Purpose	Usage	Example	Important Learning Insight
/compact	Compress conversation context	Extend usable context	Long sessions	`/compact auth changes`	Important for long workflows
/resume	Reopen prior session	Continuity	Claude Code	`/resume bugfix`	Useful for persistent projects
/clear	Start fresh conversation	Reset context	Development	`/clear`	Old sessions still recoverable
/plan	Planning mode	Structured execution	Coding workflows	`/plan refactor auth`	Planning improves reliability
/debug	Debug assistance mode	Troubleshooting	Development	`/debug login issue`	Helpful for diagnostics
/review	Review changes/PRs	Quality control	Git workflows	`/review 22`	AI review still requires humans
/memory	Manage project memory	Persistent instructions	Claude Code	`/memory`	Strong memory files improve results
/mcp	Manage MCP integrations	Tool connectivity	Claude Code	`/mcp`	Critical for advanced tooling

10. APIs & Runtime Concepts

Term	Description	Purpose	Usage	Example	Important Learning Insight
Message Batches API	Bulk asynchronous processing	Scale workloads	Embeddings/ extraction	Batch jobs	Bad for real-time chat

Term	Description	Purpose	Usage	Example	Important Learning Insight
Streaming APIs	Incremental response delivery	UX responsiveness	Assistants	Live typing	Harder orchestration
Async workflows	Non-blocking execution	Scalability	Enterprise systems	Queue-based processing	Debugging becomes harder
Latency	Response delay	UX/ performance	APIs	2s response time	Tool use increases latency
Rate limits	API usage restrictions	Infrastructure protection	SaaS APIs	RPM/TPM limits	Requires retry/ backoff logic
Concurrency	Parallel execution	Throughput	Multi-agent systems	Parallel retrieval	Coordination complexity increases
Webhooks	Event-driven callbacks	System integration	Automation	Payment events	Reliability handling important
GraphQL	Flexible API query layer	Efficient data retrieval	SaaS apps	Query exact fields	Complexity grows quickly
REST APIs	Standard HTTP APIs	System communication	Web services	GET/POST APIs	Simpler than GraphQL often

11. Safety & Governance

Term	Description	Purpose	Usage	Example	Important Learning Insight
Constitutional AI	Anthropic alignment method	Safer outputs	Claude systems	Principle-guided behavior	Central Anthropic philosophy
Safety layers	Protection mechanisms	Risk reduction	Production AI	Moderation pipelines	Safety ≠ perfect prevention

Term	Description	Purpose	Usage	Example	Important Learning Insight
Policy enforcement	Rule validation	Compliance	Enterprise AI	Content restrictions	Requires strong gates
Moderation	Harmful content filtering	Safety	User-facing systems	Toxicity checks	False positives inevitable
Human-in-the-loop	Human review involvement	Reliability/ safety	Enterprise workflows	Approval queues	Critical for high-risk domains
Guardrails	Behavioral constraints	Reliability	AI systems	Output restrictions	Weak guardrails fail silently
Permissions	Tool/action authorization	Security	Agent systems	Allow/deny rules	Important for tool safety
Sandboxing	Isolated execution environment	Risk containment	Browser/ code agents	Restricted runtime	Essential for dangerous actions

12. Browser Automation & Computer Use

Term	Description	Purpose	Usage	Example	Important Learning Insight
Playwright	Browser automation framework	Automation/ testing	AI agents	Web interaction	Browser automation is fragile
Cypress	Frontend testing framework	UI testing	QA systems	E2E tests	Flaky tests common
Browser automation	Programmatic browser control	Web workflows	Agents	Form filling	Anti-bot systems are difficult
Computer use	AI controlling computer actions	Autonomous workflows	AI assistants	Click/type/ navigation	Reliability remains hard

Term	Description	Purpose	Usage	Example	Important Learning Insight
DOM interaction	Manipulate webpage structure	Browser agents	Automation	CSS selectors	UI changes break agents
Session isolation	Separate browser contexts	Multi-user workflows	Playwright contexts	Isolated login sessions	Important for parallel workflows

13. Production AI Infrastructure

Term	Description	Purpose	Usage	Example	Important Learning Insight
Queues	Task buffering systems	Scalability	Async pipelines	RabbitMQ/ Kafka	Essential for production scale
Observability	System visibility/ monitoring	Debugging	Production AI	Logs/metrics/ traces	Critical but often ignored
Telemetry	Runtime event collection	Analytics	AI systems	Tool usage metrics	Enables optimization
Caching	Store reusable outputs	Performance/ cost	RAG/APIs	Embedding cache	Poor invalidation causes bugs
Checkpointing	Save intermediate state	Recovery	Training/ agents	Resume workflows	Needed for long jobs
Resumability	Continue interrupted tasks	Reliability	AI agents	`--resume`	Harder than people think
Distributed workflows	Multi-machine execution	Scale	Enterprise orchestration	Parallel agents	Coordination complexity explodes

Term	Description	Purpose	Usage	Example	Important Learning Insight
Orchestration layers	Systems coordinating components	Workflow control	Enterprise AI	LangGraph orchestration	Often more important than prompts

High-Value Learning Priorities for Claude Solution Architect Foundation

Highest Priority Topics

1. Tool use
2. MCP
3. Agent orchestration
4. RAG
5. Context management
6. Structured outputs
7. Evaluation systems
8. Safety and guardrails
9. Reliability engineering
10. Production AI workflows

Biggest Mistakes Beginners Make

Mistake	Reality
Obsessing over prompts only	Production AI is mostly orchestration and reliability
Thinking bigger context solves memory	Context still degrades
Believing citations guarantee truth	Retrieval can still be wrong
Assuming high coverage means quality	Coverage metrics are often misleading
Thinking multi-agent systems are magic	Most are orchestration wrappers
Overusing randomness	Enterprise systems prefer deterministic behavior
Ignoring evals	Production systems fail without evaluation pipelines
Treating tool use as trivial	Tool reliability is one of the hardest problems

Final Learning Advice

If preparing seriously for Claude Solution Architect Foundation:

- Learn systems thinking, not memorization.
- Focus on reliability and orchestration.
- Understand tradeoffs, not just definitions.
- Build small real projects:
- one RAG app
- one extraction pipeline
- one tool-using agent
- one browser automation workflow
- Read Anthropic docs and MCP docs directly.
- Study failure modes aggressively.

Most people remain at “prompt engineering” level. The certification direction is moving toward: production AI systems engineering.